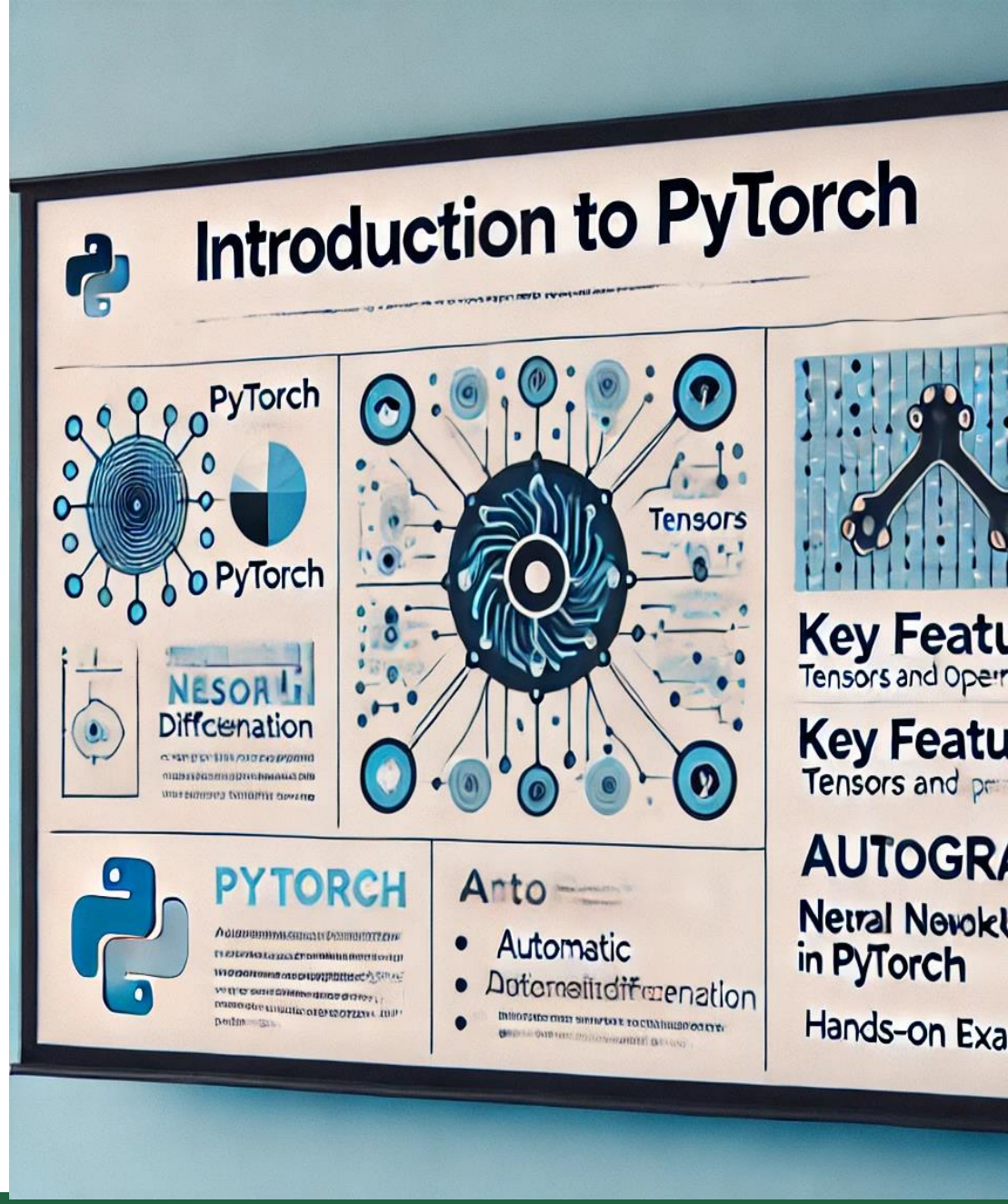
Introduction to PyTorch

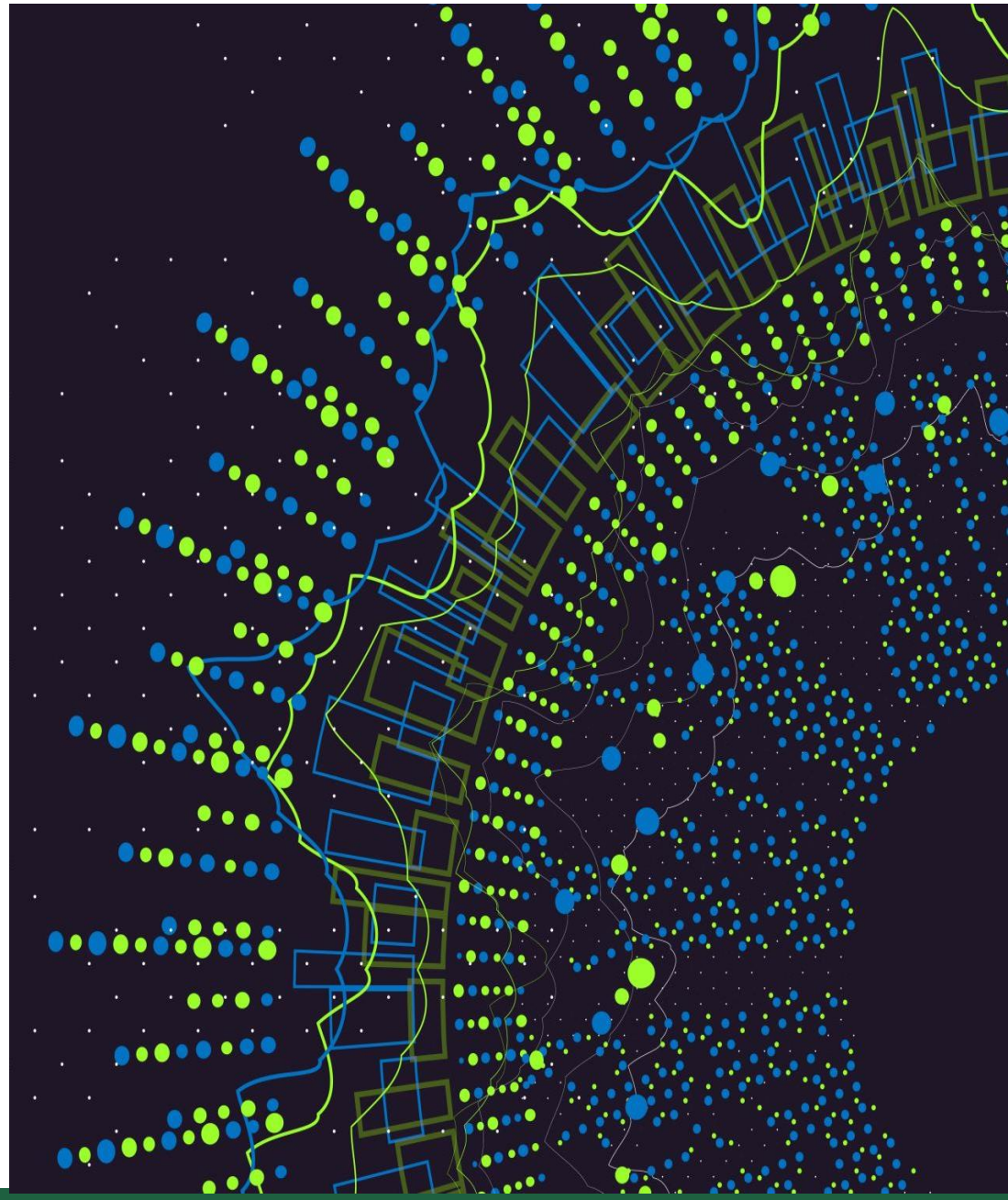
Instructor: Stephin Rachel Thomas

June 17, 2025



Today’s Topics

- What is PyTorch
- History of PyTorch
- Key Features
- PyTorch Vs Tensorflow
- Core components of PyTorch
- Deep dive into tensors
- Neural Network Module
- Optimizers and Loss Function
- Data Handling in PyTorch
- CV with PyTorch
- Advanced Features
- Community and Ecosystem
- Best Practices in PyTorch



What is PyTorch?

PyTorch is an open-source machine learning library for Python, known for its flexibility, ease of use, and dynamic computation graph.

It allows researchers to experiment quickly with deep neural networks, and it's extensively used in academia and industry for applications ranging from computer vision to natural language processing.

PyTorch is one of the most popular deep learning frameworks that allows us to implement neural network more efficiently.

History of PyTorch

Early Beginnings (Torch) - 2002

- PyTorch evolved from **Torch**, a machine learning framework built on **Lua** in 2002.
- Torch gained popularity for its **GPU acceleration** and was widely used in academia.
- However, Lua was not as popular as Python, limiting Torch's adoption.

Birth of PyTorch - 2016

- Facebook's **AI Research Lab (FAIR)** developed PyTorch to provide a **Pythonic alternative** to Torch.
- Released in **October 2016**, PyTorch introduced **dynamic computation graphs** and **easy debugging**, making it popular among researchers

Growth and Adoption (2017-2020)

- Quickly became the preferred framework for **AI research, deep learning, and NLP**.
- Hugging Face adopted PyTorch for **transformers and NLP models**.
- Facebook introduced **TorchScript** for model deployment.

Competition with TensorFlow (2021- Present)

- By 2021, PyTorch had surpassed TensorFlow in research usage.
- **PyTorch 2.0 (2023)** introduced **faster performance with torch.compile**.
- In **September 2022**, PyTorch transitioned to the **Linux Foundation**, ensuring open governance.
- Today, it's widely used in academia, industry, and production AI models.

Key Features of PyTorch

PyTorch's key features include its dynamic computation graph (which allows changes to the network on the fly), strong GPU acceleration for faster computations, and its deep integration with the Python programming language. This integration makes PyTorch not only powerful but also flexible and intuitive, offering seamless compatibility with popular Python libraries like NumPy and SciPy.

PyTorch

- Facilitates building deep learning projects
- Easily run array-based calculations
- Build dynamic neural networks
- Perform auto differentiation with a strong GPU acceleration
- Developed to process large-scale image analysis
    - Object detection
    - Segmentation
    - Classification
- Supported by all major cloud platforms
    - Amazon Web Services
    - Google Cloud Platform
    - Microsoft Azure
- Supports CPU, GPU, TPU and parallel processing

PyTorch vs. Tensorflow

Feature/Aspect	PyTorch	TensorFlow
Primary Language	Python	Python, with APIs in other languages
Computation Graphs	Dynamic (Define-by-Run)	Static (Define-and-Run)
Ease of Use	Generally considered more user-friendly and intuitive	Steeper learning curve, improved with Keras
Debugging	Easier due to dynamic graphs and Pythonic nature	More complex, requires separate tools
Performance	Comparable, with slight variations based on use case	Comparable, with optimizations for large-scale
Community & Support	Strong community, especially in research	Strong community, less popular in research
Deployment	Growing in mobile and web deployment	Extensive deployment options, including TFLite
Pre-Trained Models	Available through TorchVision, etc.	Extensive range in TensorFlow Hub
Distributed Training	Supported with PyTorch Distributed	Advanced options with TensorFlow Distributed
Integration	Seamless with Python libraries	Integrates well with TensorFlow ecosystem

Use PyTorch if: You are a beginner, doing research, or need flexibility (e.g.NLP, CV).
Use TensorFlow if: You need enterprise-level solutions, mobile deployment, or production-ready models.

Core Components of PyTorch

PyTorch comprises several core components:

- Tensors, which are similar to NumPy arrays but with GPU support
- Autograd, for automatic differentiation
- Optimizers, which abstract the optimization algorithms used to train neural networks.

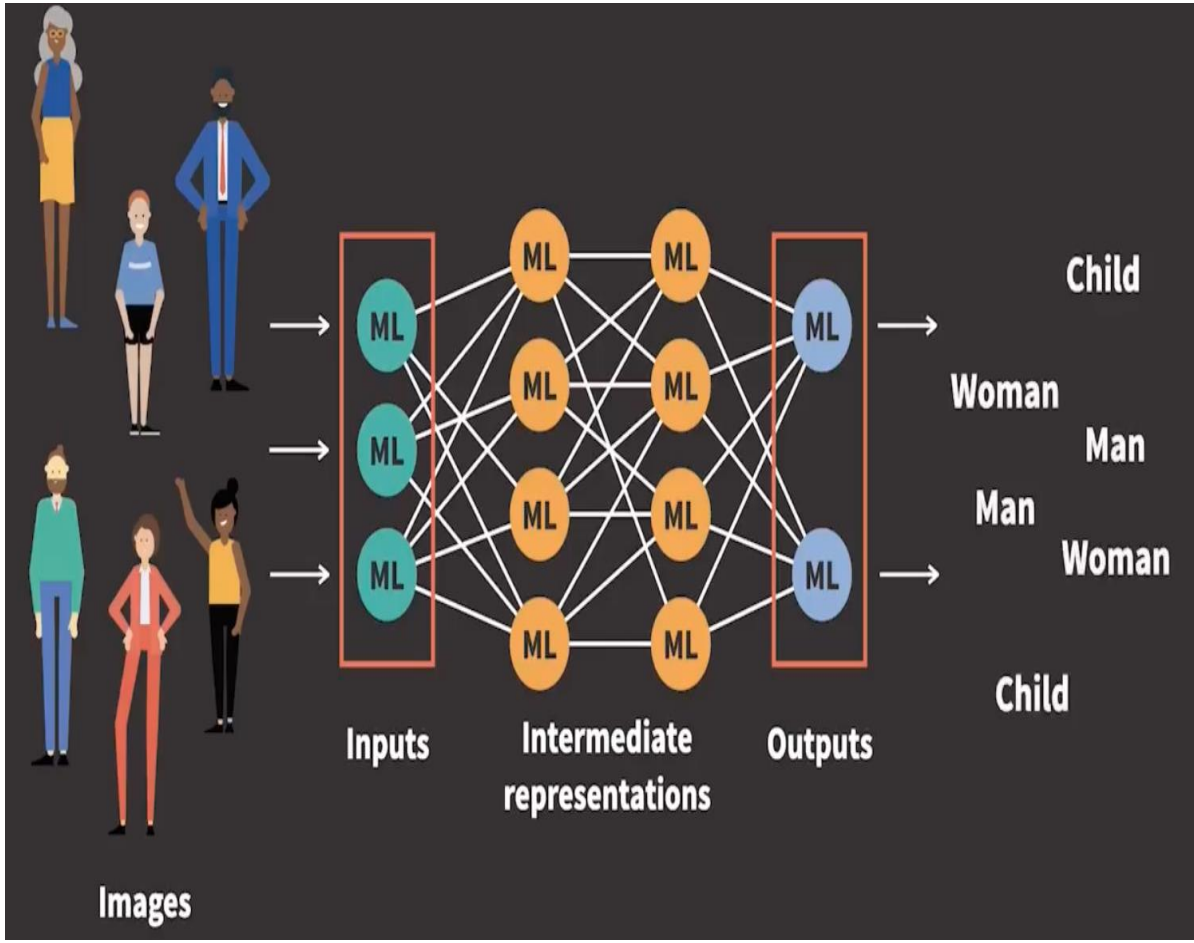
These components work together to simplify the process of creating and training complex models.

Deep Dive into Tensors

Tensors are the fundamental building blocks in PyTorch, representing data like images or text.

To handle and store the data in all stages of deep learning, PyTorch uses this essential data structure called tensor.

Inputs, intermediate representations and outputs are stored as tensors.



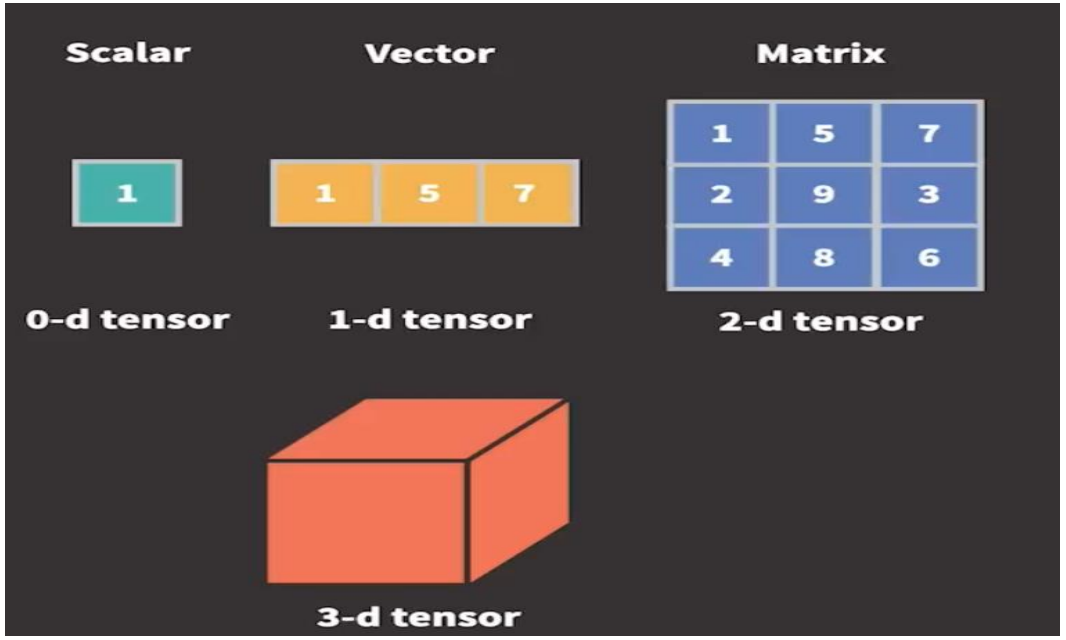
Deep Dive into Tensors

In mathematics, tensors can be defined as generalization of scalars, vectors and matrices to any dimension

In PyTorch, Tensors are multidimensional array containing elements of a single data type

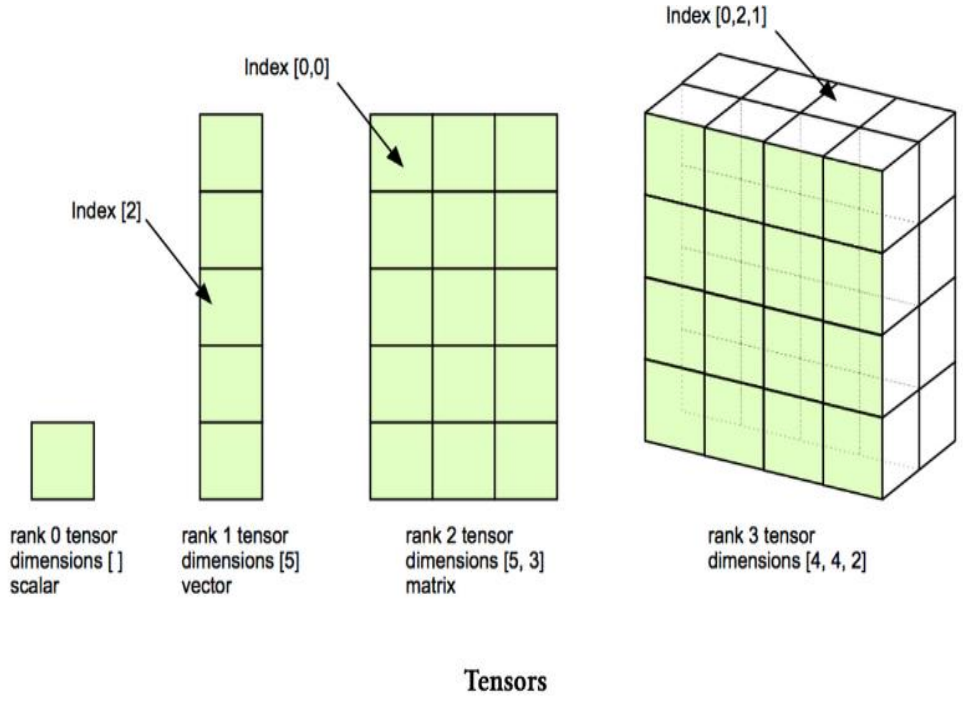
Tensor is similar to fundamental object in NumPy called ndarray

ndarray is defined as n-dimensional homogeneous array of fixed-sized items



Deep Dive into Tensors

- Tensor operations are performed significantly faster using GPUs
- Tensors can be stored and manipulated at scale using distributed processing on multiple CPUs and GPUs and across multiple servers
- Tensors keep track of the graph of computations that created them



Deep Dive into Tensors

This slide covers how tensors are created, manipulated, and used in PyTorch, with examples showing operations on tensors, and how they can be moved to a GPU for accelerated computing.

```python
import torch

# Create a tensor
tensor_a = torch.tensor([2, 3, 4], dtype=torch.float32)

# Manipulating tensor (e.g., scaling and addition)
tensor_b = tensor_a * 2 + 1  # Element-wise scaling and addition

# Using tensor in a simple computation (e.g., element-wise multiplication)
result = tensor_a * tensor_b

# Check if CUDA (GPU support) is available
if torch.cuda.is_available():
    # Move tensors to the GPU
    tensor_a = tensor_a.cuda()
    tensor_b = tensor_b.cuda()
    result = result.cuda()
    print("Tensors moved to GPU:", tensor_a, tensor_b, result)
else:
    print("CUDA is not available. Tensors are on CPU:", tensor_a, tensor_b, result)
```

```
Tensors moved to GPU: tensor([2., 3., 4.], device='cuda:0') tensor([5., 7., 9.], device='cuda:0') tensor([10., 21., 36.], device='cuda:0')
```

Automatic Differentiation (Autograd)

There are 2 steps in training neural networks,

- Forward propagation
- Backward propagation

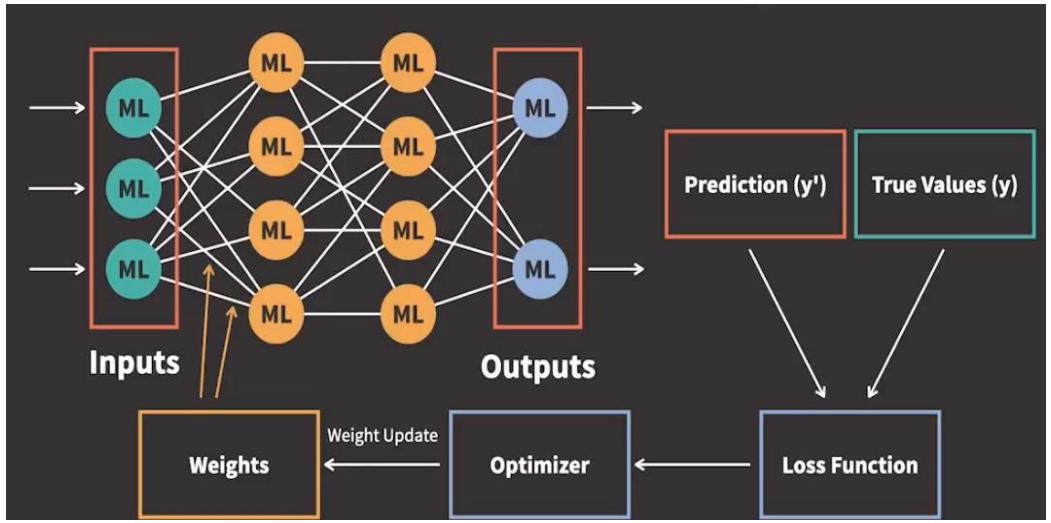
After the loss function is calculated, the derivative of the loss function in terms of the parameters are calculated

Iteratively update the weight parameters accordingly that the loss function returns the smallest possible loss

This is called iterative optimization, as we use an optimizer to perform the update of parameters

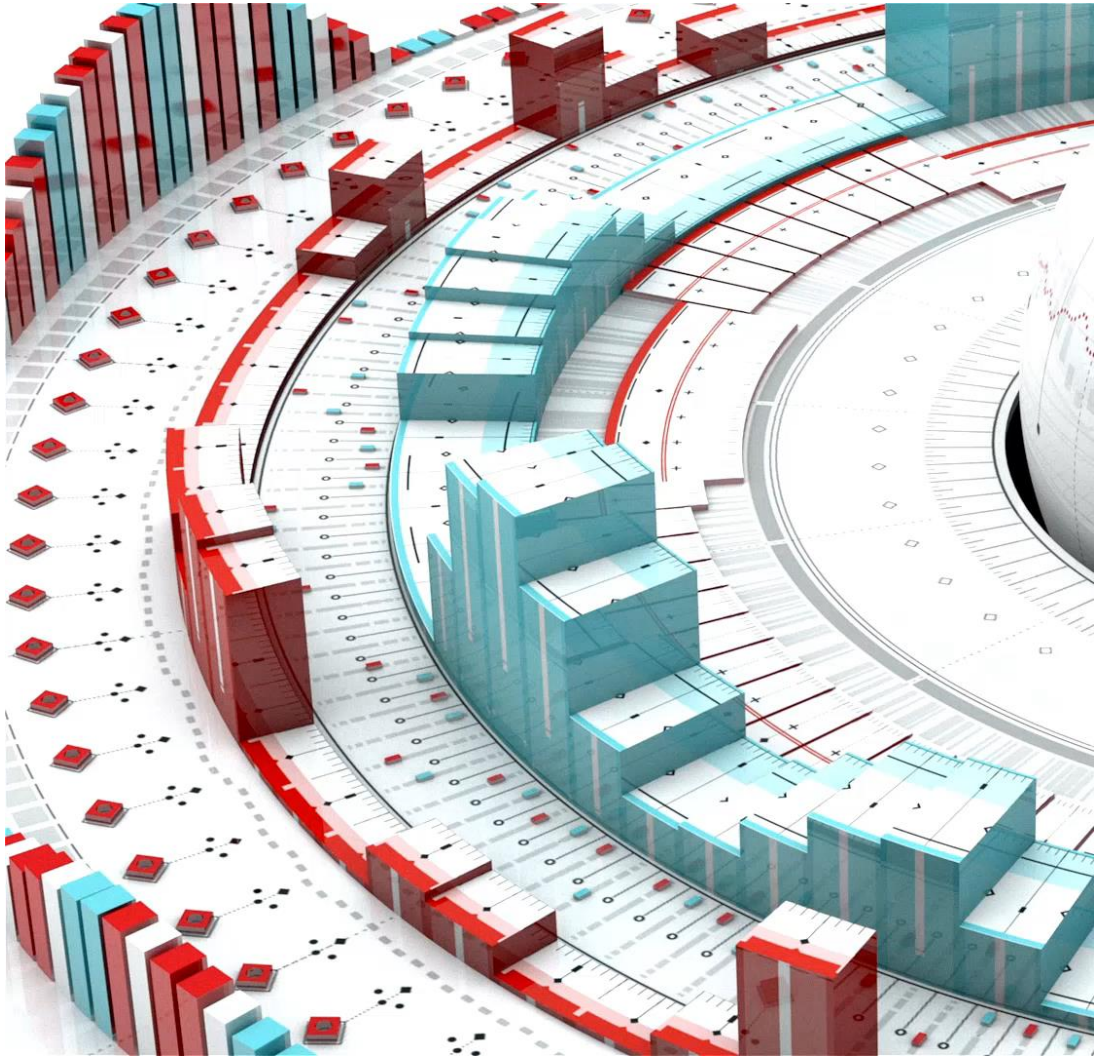
This is called gradient based optimization

Autograd is a set of techniques that allows us to compute gradients for arbitrary complex loss functions efficiently



Neural Network Module

PyTorch's nn module is a comprehensive library that includes a wide range of pre-defined layers, loss functions, and utilities that are essential for building neural networks. It provides an easy way to construct network architectures, enabling both the simple assembly of standard layers like convolutional and linear layers, and the customization of more complex models. This module greatly simplifies the process of defining a network's forward pass, with its intuitive and Pythonic approach, allowing for clear and readable code that closely resembles the actual architecture of the model.



Optimizers and Loss Functions

PyTorch offers various optimizers like SGD (Stochastic Gradient Descent), Adam, and RMSprop, each providing different approaches to navigating the loss landscape.

Key loss functions include CrossEntropyLoss, used for classification tasks, and Mean Squared Error (MSE), commonly used in regression.

These functions measure the difference between the predicted output and actual data, guiding the model's improvements during training.

```python
import torch
import torch.nn as nn

# Example tensors representing predicted outputs and actual labels
# Predicted outputs (logits) from a neural network, for a batch of 3 samples and 4 classes (unnormalized scores)
predicted_logits = torch.tensor([[1.5, 0.5, -1.2, 0.7],
                                 [1.2, 0.2, 0.5, -1.0],
                                 [0.3, -0.9, 1.0, 1.1]])

# Actual labels (indices of the correct class), for the same batch of 3 samples
actual_labels = torch.tensor([0, 1, 3])  # Assuming class indices are 0, 1, 2, 3

# Define the cross entropy loss function
loss_fn = nn.CrossEntropyLoss()

# Calculate the loss
loss = loss_fn(predicted_logits, actual_labels)

print("Cross Entropy Loss:", loss.item())
```

```
Cross Entropy Loss: 1.075467824935913
```

DL Training Process

- Data preparation

Converts generic data (text, image, video, audio etc.) to numerical values, in the form of tensors. Tensors are pre-processed during transforms and then group them into batches before passed into the model

- Model Development

It involves model design, training and testing performance

Dataset is divided into training data, validation data and testing data

- Model Deployment

Save the model to a file

Deploy the model to a product or service (usually on a cloud server or to an edge device)

Data Handling in PyTorch

PyTorch's Dataset and DataLoader classes streamline data preprocessing and loading. Dataset allows for custom data handling, while DataLoader efficiently batches and loads data, offering options like shuffling and multiprocessing. For instance, in image classification, DataLoader can automate the process of loading and transforming images into tensor format, ready for model input.

```python
import torch
from torch.utils.data import Dataset, DataLoader

class CustomDataset(Dataset):
    def __init__(self, size, num_classes):
        self.size = size
        self.num_classes = num_classes
        self.data = torch.randn(size, 10)  # Random features (size x 10)
        self.labels = torch.randint(0, num_classes, (size,))  # Random labels

    def __len__(self):
        return self.size

    def __getitem__(self, idx):
        return self.data[idx], self.labels[idx]

# Create an instance of the CustomDataset
dataset = CustomDataset(size=1000, num_classes=5)

# Create a DataLoader
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)
```

Building a Simple Neural Network

Building a neural network in PyTorch involves defining a model class that inherits from nn.Module. The class typically includes an __init__ function to define layers and a forward function for the data flow. For example, a simple network for image classification might include convolutional layers, activation functions like ReLU, and a final fully connected layer.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        # Convolutional layer (input channels, output channels, kernel size)
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1)

        # Max pooling
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2, padding=0)

        # Fully connected layers
        self.fc1 = nn.Linear(128 * 4 * 4, 512) # Assuming input images are 32x32 pixels
        self.fc2 = nn.Linear(512, 10) # Output layer for 10 classes

    def forward(self, x):
        # Apply convolutions and max pooling
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))

        # Flatten the tensor for the fully connected layers
        x = x.view(-1, 128 * 4 * 4)

        # Apply fully connected layers with ReLU and output layer
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

Computer Vision with PyTorch

PyTorch's robustness in computer vision comes from its comprehensive libraries like torchvision, which includes pre-trained models, datasets, and image transformation tools. It enables tasks such as image classification, object detection, and segmentation. For example, using a pre-trained ResNet model from torchvision, one can easily implement transfer learning for custom image classification tasks.

Advanced Features in PyTorch

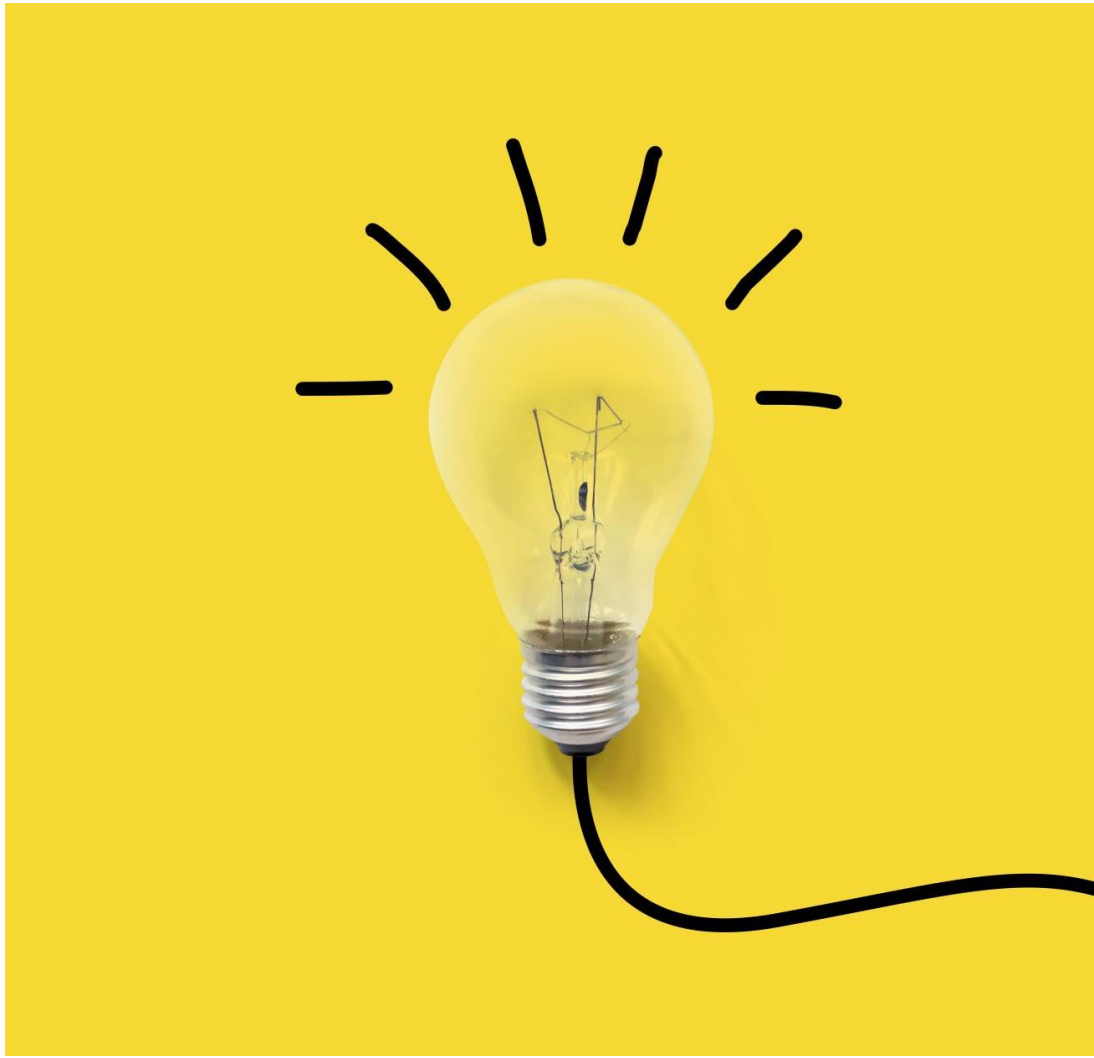
PyTorch's advanced features include support for CUDA, enabling GPU acceleration for faster computation. It also offers distributed training capabilities, essential for handling large datasets and complex models. PyTorch's JIT Compiler improves performance by converting models to optimized TorchScript, and its C++ front-end allows integration with C++ codebases, enhancing flexibility and efficiency in model deployment.

Community and Ecosystem

PyTorch is supported by a strong community of developers and researchers. The PyTorch ecosystem includes extensive documentation, tutorials, and a forum for discussions. Notable contributions come from both academic and industry leaders, ensuring continuous improvements and updates. This robust support system is invaluable for both beginners and advanced users for troubleshooting and keeping abreast of the latest developments in deep learning.

Best Practices in PyTorch

To ensure efficient and effective model training in PyTorch, it's important to follow best practices such as using GPU acceleration where possible, properly splitting data into training and validation sets, and utilizing PyTorch's inbuilt functionalities like DataLoader for data management. Regularly saving and loading models during training prevents data loss and allows for fine-tuning. Keeping code modular and well-documented enhances readability and maintainability.



Conclusion and Key Takeaways

To conclude, PyTorch stands out as a flexible, intuitive, and powerful tool for deep learning, especially in computer vision. Its dynamic nature, strong GPU support, and extensive community make it a top choice for both researchers and industry professionals. As we continue to witness rapid advancements in AI and machine learning, PyTorch is well-positioned to remain at the forefront of innovation.